



Graphic Era
Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Full Stack Authentication System

Student/Team Information

Team Name:	Code_Clash
Team member 1 (Team Lead) Name : Anshika Saklani University Roll no : 2318420 Student ID : 23012076 Email: anshikasaklani894@gmail.com	

Team member 2 Name : Ayush Chand University Roll no : 2318582 Student ID : 230112536 Email: ayushchand862@gmail.com	 A circular portrait of a young man with dark hair, wearing a light-colored sweater over a blue collared shirt. The background is a solid blue color.
Team member 3 Name : Akriti Rawat University Roll no : 2318263 Student ID : 230221541 Email: akritirawat12345@gmail.com	 A portrait of a young woman with long, dark, wavy hair, wearing a dark blue hoodie with the word 'GRAPHIC' visible. The background is a plain, light-colored wall.
Team member 4 Name : Rakhi Bisht University Roll no : 2319380 Student ID : 23012177 Email: rakhibisht24122004@gmail.com	 A portrait of a young woman with long, dark, wavy hair, wearing a dark blue blazer over a white shirt and a red and white striped tie. The background is a solid blue color.

PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

The *Full Stack Authentication System* is designed to address the critical security vulnerabilities associated with traditional **Single-Factor Authentication (SFA)**, where reliance on passwords alone creates a single point of failure. With the increasing prevalence of cyber threats such as credential stuffing and phishing attacks, there is a pressing need for more robust identity verification mechanisms.

This project aims to develop a secure, scalable, and production-ready authentication framework by implementing a multi-layered security approach. The system integrates **Two-Factor Authentication (2FA)** using **Time-based One-Time Passwords (TOTP)**, ensuring that access is granted only after verifying both user credentials and a dynamically generated code from a trusted device. This significantly reduces the risk of unauthorized access, even if passwords are compromised.

The backend is built using **Node.js and Express.js**, utilizing **stateless JSON Web Tokens (JWT)** for efficient and scalable **session management**. Passwords are securely stored using bcrypt hashing, while MongoDB is used for flexible and secure data storage. The system also incorporates Role-Based Access Control (RBAC) to enforce granular permissions and enhance security across different user roles.

Additionally, the project includes an administrative dashboard for user management and a secure recovery mechanism using hard tokens, replacing traditional email-based password reset methods. Overall, this system provides a comprehensive and modern solution for secure authentication, suitable for enterprise and high-security applications, while ensuring scalability, maintainability, and strong protection against evolving cyber threats.

Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The project follows a modular, decoupled architecture based on the principle of *defense in depth*, ensuring multiple layers of security across the authentication workflow. The system is designed as a RESTful API using Node.js and Express.js, enabling efficient handling of client requests and seamless integration with frontend applications.

The authentication process is stateless and token-based. Upon successful login with credentials, the server generates a **JSON Web Token (JWT)**, which is used for subsequent request authorization. To strengthen security, Two-Factor Authentication (2FA) is integrated **using Time-based One-Time Passwords (TOTP)**. Users must verify a dynamically generated code via Google Authenticator before access is granted, ensuring protection even if credentials are compromised.

The backend communicates with a MongoDB database using Mongoose for schema modeling and secure storage of user data, including hashed passwords and secret keys. Passwords are encrypted using bcrypt with salting to prevent brute-force attacks. For TOTP functionality, libraries such as speakeasy or otplib are used to generate and validate authentication codes.

Role-Based Access Control (RBAC) is implemented through middleware, allowing controlled access to protected routes based on user roles (e.g., admin or standard user). Every API request is validated for authentication and authorization before processing.

The frontend, built with HTML/CSS or optionally React.js, interacts with the backend via **HTTP/HTTPS** protocols, ensuring secure communication. API testing and validation are performed using Postman, while sensitive configuration data is managed through environment variables using dotenv.

Overall, the architecture ensures **scalability, maintainability, and robust protection** against modern security threats.

Tasks Completed

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Frontend Development (UI design, integration)	Akriti Rawat, Ayush Chand
Backend Development (API, authentication logic)	Anshika Saklani
Database Design & Management (MongoDB, schemas)	Rakhi Bisht
Password Security using bcrypt	Ayush Chand, Rakhi Bisht
API Testing & Validation using Postman	Ayush Chand, Rakhi Bisht

Challenges/Roadblocks

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

One of the primary challenges faced during development was **the implementation of secure Two-Factor Authentication (2FA)**, particularly related to email-based verification issues. While integrating 2FA, we initially explored sending One-Time Passwords (OTPs) via email. However, this approach led to several practical problems such as delayed email delivery, spam filtering, and dependency on external SMTP services. In some cases, OTPs expired before reaching the user, negatively affecting the user experience and reliability of authentication.

To address this, we shifted to a more robust solution using Time-based One-Time Passwords (TOTP) with authenticator apps. This eliminates dependency on email services and ensures real-time code generation on the user's device, improving both security and usability.

Tasks Pending

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
UI enhancements and responsiveness improvements in the frontend	Akriti Rawat
Complete testing of all edge cases in 2FA flow (invalid/expired TOTP)	Anshika Saklani
Optimization of API response times and error handling	Rakhi Bisht
Final security audit (checking vulnerabilities like token misuse, rate limiting)	Ayush Chand

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The project delivers a secure full-stack authentication system using Node.js and Express.js. It includes password protection with bcrypt and stateless authentication using JWT. A key feature is Two-Factor Authentication (2FA) using TOTP for enhanced security. Role-Based Access Control (RBAC) is implemented to manage user and admin permissions, along with an admin interface. Additional deliverables include a MongoDB database schema, tested REST APIs using Postman, and deployment configuration using environment variables. Overall, the system is scalable, secure, and suitable for real-world applications.

Progress Overview

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

The project is about **70–80% complete**. Core backend features like JWT authentication, bcrypt password security, and TOTP-based 2FA are implemented. Database setup and RBAC are also functional.

Frontend development is partially done, with some UI improvements pending. A minor delay occurred due to issues with email-based OTP, which was replaced by a more reliable TOTP system. The project is on track, with remaining work including frontend polishing, testing, documentation, and deployment.

Codebase Information

(Repository link, branch, and information about important commits.)

Codebase Information

- **Repository Link:** <https://github.com/Anshika-111105/Full-stack-Auth-System>
- **Branch:** main (primary development branch)

Important Commits

- **Initial Project Setup:** Created project structure with Node.js, Express.js, and basic server configuration
- **Authentication Module:** Implemented user signup/login with JWT-based authentication
- **Password Security:** Integrated bcrypt for hashing and salting passwords
- **2FA Integration:** Added TOTP-based Two-Factor Authentication using speakeasy/otplib
- **Database Setup:** Designed MongoDB schemas using Mongoose
- **RBAC Implementation:** Added middleware for Role-Based Access Control
- **API Testing:** Tested endpoints using Postman and handled error cases

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
User Registration & Login API	Pass	Successfully tested with valid and invalid inputs
JWT Authentication & Authorization	Pass	Token generation and protected routes working correctly
Password Hashing (bcrypt)	Pass	Passwords securely hashed and verified
Two-Factor Authentication (TOTP)	Pass	Codes generated and validated correctly via authenticator app
API Testing using Postman	Pass	All endpoints tested with proper responses
Error Handling & Edge Cases	Partial Pass	Minor edge cases (expired tokens, invalid TOTP) under testing

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

- Backend Authentication System (Node.js, Express, JWT): Completed - Password Security (bcrypt hashing): Completed - Two-Factor Authentication (TOTP Integration): Completed - Database Design (MongoDB with Mongoose): Completed - Role-Based Access Control (RBAC): Completed - Admin Dashboard: In Progress - Frontend Interface (UI & Integration): In Progress - API Testing (Postman): Completed - Documentation (API & Schema): In Progress - Deployment & Environment Setup: Pending
--

Overall, major core functionalities are completed, with remaining work focused on frontend, documentation, and deployment.